
UNIT 6: PLOTTING AND VISUALISATION USING PYTHON

Structure	Page No.
6.0 Introduction	139
6.1 Objectives	140
6.2 Data Visualisation and Python	140
6.2.1 Figures and Subplots	142
6.3 Customisation of Chart Elements and Layout	146
6.3.1 Colours, Markers, Line Styles, Ticks and Legends	146
6.3.2 Annotations	148
6.4 Saving Plots to a File	151
6.5 Use of Configuration in Matplotlib	154
6.6 Plotting with pandas and seaborn	155
6.7 Making Different Types of Plots	157
6.7.1 Line Plots	157
6.7.2 Bar Plots	158
6.7.3 Histograms and Density Plots	159
6.7.4 Scatter or Point Plots	160
6.7.5 Facet Grids and Categorical Data	161
6.8 Summary	162
6.9 Solutions/Answers	162

6.0 INTRODUCTION

In this course, so far we have discussed about the data wrangling in detail. Various aspects of data wrangling, such as data profiling, data cleaning, data transformation, combining and reshaping data sets and data aggregation have been discussed. From this and later units, we will be focusing on data visualisation.

This unit provides a detailed discussion on the use of matplotlib library with the help of examples. The unit does not try to cover all the functionality of this library, as it is too extensive. Rather, it covers some of the important aspects of plotting with the help of a library. The unit also discusses how you can make several subplots within a Figure using matplotlib. This is followed by examples of use of chart elements and layout, such as line style, markers, legends, etc. This is followed by a discussion on creating axis-labels and chart title. The unit also shows the example of creating annotations in a graph and saving data to file. This is followed by discussion on the process of creating different kinds of plots like line charts, bar charts, histograms, scatter plots, etc.

Thus, this unit provides an introduction to the process of creating graphs using matplotlib and few other libraries. Please note that Python has large number of libraries that can be used to draw plots. Covering all these is beyond the scope of this unit.

6.1 OBJECTIVES

After going through this unit, you should be able to:

- appreciate the need of data visualisation and plotting using matplotlib library of Python;
- write Python code to create charts with titles, axis-labels, legends, colours, ticks, annotations, etc.;
- create more than one plots in a figure;
- save plot as a file in png or pdf or other picture format;
- appreciate the importance of using configurations;
- demonstrate the use of seaborn library for plotting;
- making different kinds of plots like line plot, bar plot, histogram, scatter plot and facet grid.

6.2 DATA VISUALISATION AND PYTHON

As discussed in previous units, data wrangling is used to prepare data for analysis. However, before you start analysing data, you may like to represent the data in a graphical form that is easy to interpret. Data visualisation allows you to prepare charts from your data with the purpose of identifying basic characteristics of the data so that you can select suitable analytical methods for that data. Visualisation helps to communicate information to the users more effectively, provided the correct form of visualisation is used.

One of the libraries of Python that is suitable for data visualisation is matplotlib. The matplotlib library in Python, an open-source library, allows you to create various types of plots, including line graphs, pie charts, scatter plots, and more.

The complete collection of source code, configuration files, and other related materials for the matplotlib library is available on GitHub at the link: <https://github.com/matplotlib/matplotlib>

One of the key sub-modules of the matplotlib library is pyplot. We will be using this extensively in this section. Before we start using the matplotlib library, please note that the following code is required to run the matplotlib functions if you are NOT using any graphical user interface (GUI), i.e. you are using a command line interface. These lines are used in the first program given in Figure 2. However, if you are working on Google Colab, you should omit these three lines. For all subsequent programs, these lines are omitted.

```
#First three lines for drawing Graphs on standard output if not using a GUI
import sys
import matplotlib
matplotlib.use('Agg')
```

The first line imports the ‘sys’ module, which contains system-specific functions. One of the main functions required by the matplotlib library is the standard output function, such as `sys.stdout.buffer` function. The second line imports the matplotlib library itself. The third line refers to a backend system named ‘Anti-grain-geometry’(Agg), which converts the plots to storage file formats like .png, .pdf, etc.

In addition, the final two lines of these programs should contain the following lines: if we are using a non-GUI environment, use the following two lines. On Google Colab, they will not be required, so except for the Figure 2 program, they are omitted.

```
#Last two lines for making the drawing on the standard output if not using GUI
plt.savefig(sys.stdout.buffer)
sys.stdout.flush()
```

The first of these lines (`plt.savefig(...)`) causes the saving of a figure to a file, and displays it on the standard output. The second line displays the content of the buffer on the standard output device immediately. For more details on these aspects, you may refer to online content on Python Programming.

Let us explain how the matplotlib library can be used to generate plots. The following example creates a bar chart for the data given in Figure 1.

Index #	Name	Department	Specialisation	Salary	YearsWorking
0	Arav S	Design	Web Development	100000	3.5
1	Ravi M	Database	SQL	200000	2.5
2	Rehan D	Design	SQL	150000	1.5
3	Ben A	Database	Web Development	200000	2.0
4	Rai Y	Design	SQL	100000	1.0

Figure 1: Sample Data (Same as Unit 5)

```
import pandas as pd
# Part (a): Code to Create the data frame Employee_data
empdata = {'Name': ['Arav S', 'Ravi M', 'Rehan D', 'Ben A',
                    'Rai Y'],
            'Department': ['Design', 'Database', 'Design', 'Database',
                           'Design'],
            'Specialisation': ['Web Development', 'SQL', 'SQL', 'Web
Development', 'SQL'],
            'Salary': [100000, 200000, 150000, 200000, 100000],
            'YearsWorking': [3.5, 2.5, 1.5, 2.0, 1.0]}
#Creating a data frame of the data of employees
Employee_data = pd.DataFrame(empdata)
```

```
#Part (b): First three lines for drawing Graphs on standard
output
import sys
import matplotlib
matplotlib.use('Agg')
```

```
#Part (c): Make two lists and use matplotlib code to make a bar
chart
import matplotlib.pyplot as plt
name_list = Employee_data['Name'].tolist()
salary_list = Employee_data['Salary'].tolist()
plt.bar(name_list, salary_list)
plt.show()
```

```
#Part (Final): Last two lines for making the drawing on the
screen
plt.savefig(sys.stdout.buffer)
sys.stdout.flush()
```

Figure 2: Code for creating a Bar chart of Name at the x-axis and Salary at the y-axis.

The output of Figure 2 is shown in Figure 3. Please note the following points about the code:

1. Figure 2, Part(a) creates the data frame Employee-data. In subsequent programs in this unit, we will not rewire this code, but will refer to it as “Code of Figure 2, Part (a)”.
2. Figure 2, Part (b) and Figure 2, Part(Final) are used to display the output on the standard output. These codes are already explained in this section. We will not require these codes in subsequent programs.
3. Part (c) of Figure 2 contains the code for displaying the bar chart. The first line of this code is a comment, the second line imports the *pyplot* module of the *matplotlib* library as a user-defined object name *plt*. You can use any other name too, but *plt* is one of the most popular convention. The next two lines create the lists for ‘Name’ and ‘Salary’ from the Employee_data dataframe. These two lists, viz. name_list and salary_list, are for the x-axis and y-axis data of the bar chart, which is plotted using *.bar* method with the two lists as parameters. This bar chart is then displayed using the *.show* method. Figure 3 displays the output of program given in Figure 2.

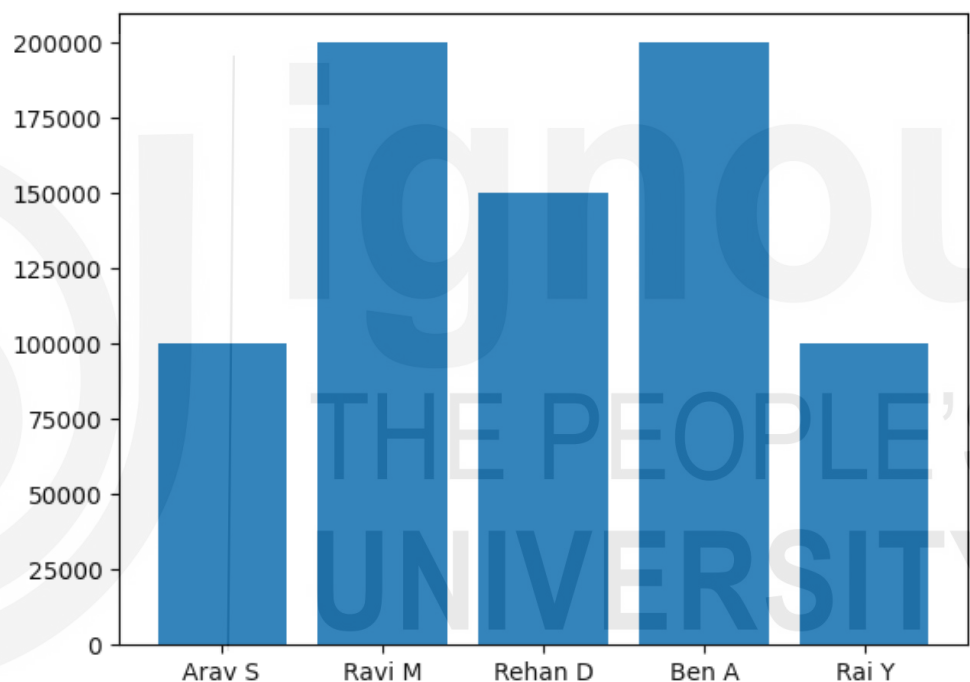


Figure 3: Output of Program in Figure 2

You may observe that Figure 3 is very basic and does not have any axis title or chart title. Further, you may like to create several subplots in a chart. The next sub-section explains these topics.

6.2.1 Figures and Subplots

In certain cases, you would like to plot several sub-plots in a diagram or Figure. This can be achieved in Python by using figure, which defines a canvas. In a single canvas, you can create several sub-plots. We demonstrate this with the help of an example. Consider you want to draw following four sub-plots in a single canvas or figure:

- i) A scatter plot between YearsWorking and Salary.
 - ii) A bar chart of Department and Total Salary
 - iii) A pie chart on Specialisation and Total Salary and
 - iv) A line chart on Department and Average of YearsWorking.
- The code for this is shown in Figure 4.

```
import pandas as pd
# Part (0): Code to Create the data frame Employee_data
empdata = {'Name': ['Arav S', 'Ravi M', 'Rehan D', 'Ben A',
                    'Rai Y'],
            'Department': ['Design', 'Database', 'Design', 'Database',
                           'Design'],
            'Specialisation': ['Web Development', 'SQL', 'SQL', 'Web
Development', 'SQL'],
            'Salary': [100000, 200000, 150000, 200000, 100000],
            'YearsWorking': [3.5, 2.5, 1.5, 2.0, 1.0]}
#Creating a data frame of the data of employees
Employee_data = pd.DataFrame(empdata)
```

```
#Part (a): Import matplotlib and create a Canvas of 2 by 2
subplots of 10 * 10 inches
import matplotlib.pyplot as plt
canvas, subplot = plt.subplots(2, 2, figsize=(10, 10))
```

```
# Part (b): Top-Left Corner: A Scatter plot betweenYearsWorking
and Salary
# Create the lists for X-axis and Y-axis from data frame
yrswork_list = Employee_data['YearsWorking'].tolist()
salary_list = Employee_data['Salary'].tolist()
#Draw the subplot with Title and Axis Labels.
subplot [0, 0].scatter(yrswork_list, salary_list)
subplot [0, 0].set_title("Scatter plot of Salary vs Years
worked")
subplot [0, 0].set_xlabel("Years Worked")
subplot [0, 0].set_ylabel ("Salary")
```

```
# Part (c): Top-right Corner: A bar chart between Department
and Total Salary
# Create the lists for X-axis and Y-axis by grouping over
Department
GroupOnDept=Employee_data.groupby('Department')
TotalSalFrame=GroupOnDept['Salary'].sum()
department_list = TotalSalFrame.index.tolist()
sumsalary_list = TotalSalFrame.values.tolist()
#Draw the subplot with Title and Axis Labels.
subplot [0, 1].bar(department_list, sumsalary_list)
subplot [0, 1].set_title("Bar chart of Department vs Total
Salary")
subplot [0, 1].set_xlabel("Department")
subplot [0, 1].set_ylabel ("Total Salary in INR")
```

```
# Part (d): Bottom-Left: A pie chart on Specialisation and
Total Salary
# Create the data and labels for the pie chart
SpecialisationGroup=Employee_data.groupby('Specialisation')
TotalSalOnSpecialFrame=SpecialisationGroup['Salary'].sum()
labelslist = TotalSalOnSpecialFrame.index.tolist()
slicesofsumsalary = TotalSalOnSpecialFrame.values.tolist()
#Draw the subplot with Title and Labels.
subplot [1, 0].pie(slicesofsumsalary , labels = labelslist, )
subplot [1, 0].set_title("Pie Chart of Total Salary for
Different Specialisations")
```

```
# Part (e): Bottom-Right: A line chart on Department and
Average of YearsWorking.
# Create the lists for X-axis and Y-axis by grouping over
Department
DeptGroup=Employee_data.groupby('Department')
```

```
AvgExpFrame=DeptGroup['YearsWorking'].mean()
dept_list = AvgExpFrame.index.tolist()
sumsalary_list = AvgExpFrame.values.tolist()
#Draw the subplot with Title and Axis Labels.
subplot [1, 1].plot(dept_list, sumsalary_list)
subplot [1, 1].set_title("Line chart of Department vs Average
Years Worked")
subplot [1, 1].set_xlabel("Department")
subplot [1, 1].set_ylabel ("Average Years Worked")

# Part (f): Create a Non-overlapping Layout
plt.tight_layout()
# Show all the subplots of the canvas
plt.show()
```

Figure 4: Creating Subplots in a Canvas or Figure

The code given in Figure 4 makes a canvas of subplots. The key points of this program are:

1. Part (0) of Figure 4 contains the code for creating a data frame named `Employee_data`.
2. Part (a) of Figure 4 imports the `pyplot` module of `matplotlib` library. It also creates a canvas or figure, which can contain one or more subplots. Please note the statement: `canvas, subplot = plt.subplots(2, 2, figsize=(10, 10))`, creates an object `canvas` consisting of objects named `subplot` using the method `.subplots(...)` of `pyplot`. The parameter `(2, 2, figsize=(10, 10))` defines a 2 by 2 matrix of subplots in the canvas object. It also declares the size of the canvas object to be of 10 inch width and 10 inch height.
3. Once you have created the canvas, the next task is to create the details of subplots which are to be displayed in the canvas. Part (b) of Figure 4 defines the first of these subplots. First, the data of the subplot is created as lists from the data frame `Employee_data` using `.tolist()` method. Please note that in this part, `subplot` object indexed by `[0, 0]` is created. Please note that since we have created 2*2 canvas so the objects of `subplot` would have indexes `[0, 0]`, `[0,1]`, `[1,0]` and `[1,1]`. You may refer to Part (c), (d) and (e) of Figure 4 to check these indexes. This part (b) creates a scatter plot, with title and axis labels. The output of this code is the top left corner scatter plot of Figure 5.
4. Part (c) of Figure 4 shows the code for the subplot to be displayed in the top right corner. The data for this box plot is created using grouping over the department, and taking the sum of the salaries of each subgroup. The lists so generated will contain the total salary of the two departments, 'Database' and 'Design'. Please note the indexes `[0,1]` of the `subplot` and the `bar(...)` method for creating a bar chart. Also, please note that the chart has a title and axis labels. The output of this code is displayed in the top right corner of Figure 5.
5. Part (d) of Figure 4 creates a pie chart in the bottom left corner of the canvas. Please note that the pie chart is being produced for grouped data on specialisation. The method used to draw pie chart is `pie(...)`. We have given two parameters to this method – the first parameter is used for computing the slice of the pie chart, and the second contains the label of those slices. Figure 5, bottom left, shows the output of this code.
6. Part(e) of Figure 4 shows the code for the fourth subplot. It uses `group by` functions to create grouped data and uses this to create a line chart with a title and axis labels. Figure 5, bottom-right chart displays this chart.
7. Part(f) of Figure 4 first selects a layout called tight layout, which helps in rendering the subplots without any overlapping. Finally, the code `plt.show()` shows all the subplots created in the canvas.

Let us summarise the process of creating a plot using the `matplotlib` library.

1. Import the *pyplot* module of *matplotlib* library using the following command:
import matplotlib.pyplot as plt
2. Prepare the data using various methods; we have demonstrated how you can create data from a dataframe.
3. You can now create a canvas consisting of one or many subplots by using the method *plt.subplots()*
4. Now, plot the graphs or charts using *plot()*, *scatter()*, *bar()*, *pie()*, etc. methods and append chart title and axis labels using *set_title()*, *set_xlabel()*, *set_ylabel()* methods.
5. Finally, show the entire canvas using *plt.show()* method.

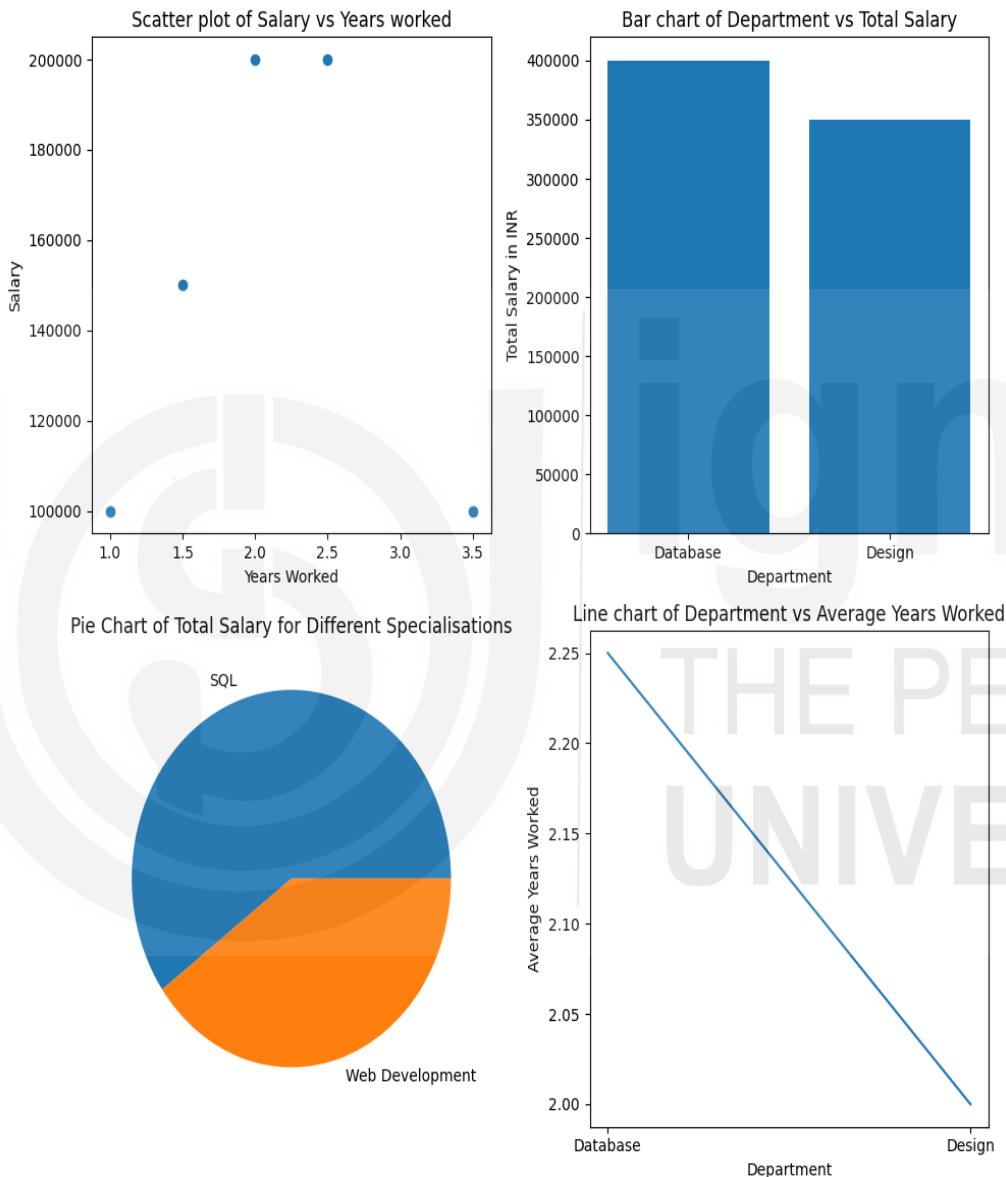


Figure 5: Output plots of the program of Figure 4

Now, you can create a canvas of subplots. Let us now learn additional methods to add more features to a graph.

Check Your Progress 1

1. List the set of commands which are essential to make a bar chart.

.....

.....

.....

2. How can you show more than one subplot in a single figure or plot?

3. How can you make a plot with a title and axis labels?

6.3 CUSTOMISATION OF CHART ELEMENTS AND LAYOUT

Data visualisation is an art of representing data in a form that can help interpret the basic characteristics of data. Therefore, a graph should have proper layout, labels, colour, etc. so that it can be interpreted effectively. This section explains how the matplotlib library can be used to add different features to a graph.

6.3.1 Colours, Markers, Line Styles, Ticks and Legends

Colours are useful in highlighting information in a graph where you want to draw the attention of a person. Markers, on the other hand, point to an exact location in a graph. And line styles can be used to distinguish the importance of different types of lines in a graph. Ticks are useful guides if your scale of display is very large. Legends are useful in communicating the meaning of different symbols or lines used in a chart. Figure 6 illustrates the use of these with the help of a program. Figure 7 shows the output of Figure 6.

```
import pandas as pd
# Part (0): Code to Create a modified data frame Employee_data
with only two columns
empdata = {'Department': ['Web Design', 'Database', 'Design',
'Database', 'Design', 'Web Design', 'Design', 'Database', 'Web
Design', 'Database'], 'Salary': [100000, 190000, 150000, 200000,
110000, 120000, 160000, 130000, 170000, 140000]}
#Creating a data frame of the data of employees
Employee_data = pd.DataFrame(empdata)
```

```
#Part (a): Import matplotlib and create a Canvas of just one
subplot of 10 inches by 6 inches
import matplotlib.pyplot as plt
canvas, subplot = plt.subplots(figsize=(10, 6))
```

```
# Part (b): Computing and displaying mean, minimum and maximum
salary.
GroupOnDept=(Employee_data.groupby('Department')['Salary'].agg(
meansal='mean', maxsal='max', minsal='min'))
print(GroupOnDept)
# Creating the list for displaying department, mean,
minimum, and maximum salaries.
department_list = GroupOnDept.index.tolist() #use of index
for department
meansalary_list = GroupOnDept['meansal'].tolist() #create the
aggregate lists
maxsalary_list = GroupOnDept['maxsal'].tolist()
minsalary_list = GroupOnDept['minsal'].tolist()
```



```

#Part (c.1): Plot the first line with colour, line style, and
markers and Label for legend.
subplot.plot(department_list, meansalary_list,
color='green',          # Line colour will be green
linestyle='dashed',     # TheLine style will be dashed
marker='o',             # Marker style as circle
markerfacecolor='yellow', # Marker fill colour as yellow
markersize=10,          # Size of the Marker
label='Mean Salary')    #Legend will show this label
# Part (c.2): Plot the second line using a shorthand format
string:
subplot.plot(department_list, maxsalary_list, 'r-.*',
label='Maximum Salary')
#In 'r-.*' 'r' means red colour of line, '-' Means a dash-dot
line, '*' is a star marker.

# Part (c.3): Plot the third line using a shorthand format
string:
subplot.plot(department_list, minsalary_list, 'b-s',
label='Minimum Salary')
#In 'b-s' 'b' means blue colour of line, '-' Means a continuous
line, 's' is a square marker.
# Part (c.4): Now, adding the chart title, the axis labels,
Minor Ticks and legend
subplot.set_title("Line chart of Department vs Salary")
subplot.set_xlabel("Department")
subplot.set_ylabel("Salary")
# Enable minor ticks first and then create ticks of desired
length on the desired axis
subplot.minorticks_on()
subplot.tick_params(axis='y', which='minor', length=4)
subplot.tick_params(axis='y', which='major', length=8)
subplot.legend()

# Now, display the plot using plt
plt.show()

```

Figure 6: Program using Colours, Line Styles, Markers and Legends

Program 5 is explained below:

1. Part (0) creates the data frame consisting of department and salary of people working in those departments. Some data has been added.
2. Part (a) of Figure 6 imports the *pyplot* module as *plt* object, which is used subsequently to create an object of figure called *canvas* and an object named *subplot* that defines the plotting details.
3. Part (b) of Figure 6 creates the list of data that is to be plotted. Please note the use of *groupby* method on Department with three distinct aggregate functions, mean, min and max. Please also note that 'Department' also forms the index of aggregated data. The variable names for aggregating data are meansal, minsal and maxsal, which are then used to create the lists of aggregated data. The entire aggregated data is shown in Figure 7(a).
4. Part (c) of Figure 6 is divided into four subparts. Part (c.1) shows how the *subplot* object is used to create the first list of departments and the mean salary in each department. In addition, this code sets the colour of the line, style of the line, marker style, marker fill colour and marker size. The label field is created for displaying in the legend.
5. Part (c.2) creates another line in the same *subplot* object. However, it uses a shortcut method of communicating the line colour, line style and marker. They are explained in the comment. Please note that *pyplot* module uses a large number of shorthand codes. You can refer to these codes from the further readings.

6. Part (c.3) creates the third line. This code also uses the shortcut method of line colour, line style and marker.
7. Part (c.4) further adds the features to the *subplot* object. This includes setting up of title, x-axis label and y-axis label. In addition, this code first enables the minor ticks, and displays the minor ticks of length 4 and major ticks of length 8. You may observe these ticks in the y-axis scale of Figure 7(b). You may compare it with the top left scatter plot y-axis in Figure 5, where only the major tick is displayed. Finally, this part shows the legend, which is visible in the top right portion of the chart in Figure 7.

	meansal	maxsal	minsalsal
Department			
Database	165000.0	200000	130000
Design	140000.0	160000	110000
Web Design	130000.0	170000	100000

Figure 7(a): The list of aggregated Data of the Program of Figure 6

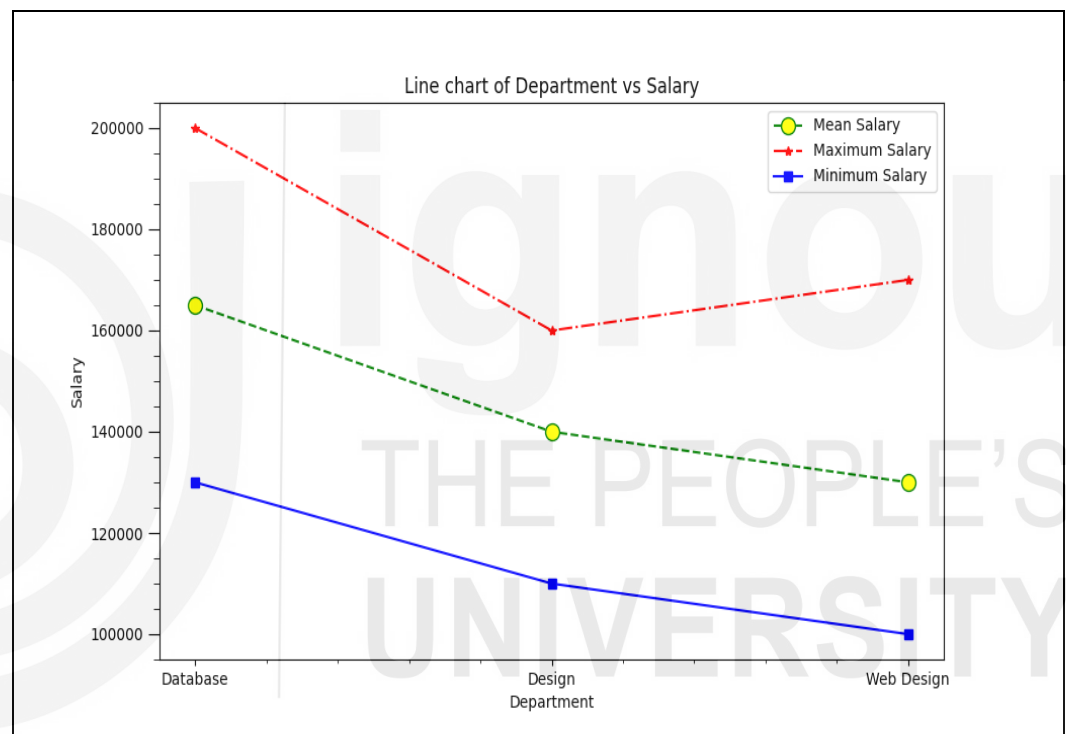


Figure 7(b): Chart Output of the Program of Figure 6

6.3.2 Annotations

Annotations can be used to highlight some of the key points of a graph. The main purpose is to draw attention to certain important aspects of the graphs. Annotations help in increasing the clarity of information being represented by graphs. How do you annotate a graph? There are several ways of annotating a graph, such as you can add an arrow or a label, etc. An annotation can be added to a graph using the following function:

```
annotate("textTODisplay", xy, xytext, arrowprops)
```

There are four basic parameters of an annotation. These are:

- "textTODisplay" parameter contains the text that is to be displayed as annotation;
- xy defines the data point for which the annotation is to be displayed;
- xytext defines the position at which annotation text is to be displayed, and

- arrowprops can be used to change the characteristics of the arrow if it is to be shown in an annotation.

The following program creates two subplots in a canvas. The first subplot is a bar chart of employee salary, as given in Figure 1, with a proper title and axis labels. This subplot also annotates the maximum salary. Please note that two person draw maximum salary, so our program should annotate both the instances. The second subplot is a department-wise line plot of total salary. In this subplot the minimum value and the line itself are annotated. Figure 8 shows the program for the graph and the annotation.

```
import pandas as pd
# Part (0): Code to Create the data frame Employee_data
empdata = {'Name': ['Arav S', 'Ravi M', 'Rehan D', 'Ben A',
'Rai Y'],
'Department': ['Design', 'Database', 'Design', 'Database',
'Design'],
'Specialisation': ['Web Development', 'SQL', 'SQL', 'Web
Development', 'SQL'],
'Salary': [100000, 200000, 150000, 200000, 100000],
'YearsWorking': [3.5, 2.5, 1.5, 2.0, 1.0]}
#Creating a data frame of the data of employees
Employee_data = pd.DataFrame(empdata)
```

```
#Part (a): Create a Canvas of 2 subplots of 12 inches width by
16 inches length
import matplotlib.pyplot as plt
canvas, subplot = plt.subplots(2, figsize=(10, 16))
```

```
# Part (b): Prepare the data to create the bar plot in subplot
1
name_list = Employee_data['Name'].tolist()
salary_list = Employee_data['Salary'].tolist()
subplot[0].bar(name_list, salary_list)
```

```
# Part (c): Find maximum salary and find all employees drawing
maximum salary
maximum_salary = Employee_data['Salary'].max()
Maximum_Sal_emp_data = Employee_data[Employee_data['Salary'] ==
maximum_salary]
```

```
# Part (d): Annotate bars for every employee who is drawing the
maximum salary
for ind_max_sal in
Maximum_Sal_emp_data.index: subplot[0].annotate( "Maximum
Salary",
xy=(name_list[ind_max_sal], salary_list[ind_max_sal]),
xytext=(-50, 0),
textcoords="offset points",
ha='right', va='bottom',
arrowprops=dict(arrowstyle="->")
)
#The command above indicates point Location, text Location, and
arrow Characteristics
# Add Title and labels, please note the use of font size
subplot[0].set_title("Bar Chart of Name vs Salary with
Annotation", fontsize=20)
subplot[0].set_xlabel("Name", fontsize=16)
subplot[0].set_ylabel("Salary", fontsize=16)
```

```
# Part (e): Subplot 2 is a line chart between Department and
Total Salary
# Create the lists for X-axis and Y-axis by grouping over
Department
```

```
GroupOnDept=Employee_data.groupby('Department')
TotalSalFrame=GroupOnDept['Salary'].sum()
department_list = TotalSalFrame.index.tolist()
sumsalary_list = TotalSalFrame.values.tolist()
#Draw the subplot with Title and Axis Labels.
subplot [1].plot(department_list, sumsalary_list, marker='o')
subplot [1].set_title("Line chart of Department vs Total
Salary")
subplot [1].set_xlabel("Department")
subplot [1].set_ylabel ("Total Salary in INR")
#Find the location of Minimum Total Salary for Annotation
min_sumsal = min(sumsalary_list)
xofmin_sumsal =
department_list[sumsalary_list.index(min_sumsal)]
subplot[1].annotate(
    f'Minimum: {min_sumsal}',
    xy=(xofmin_sumsal, min_sumsal),
    xytext=(xofmin_sumsal,min_sumsal),
    arrowprops=dict(facecolor='black', shrink=0.05)
)
#Display a text annotation at the middle of the line of the
line plot
subplot[1].text(0.5, 0.5, "Total Salary", fontsize=10,
transform=subplot[1].transAxes)
plt.show()
```

Figure 8: Different Annotations

The description of the program in Figure 8 is as follows:

1. Part (0) of the program creates the dataframe.
2. Part (a) creates a canvas for 2 subplots in a plot of the width 10 inches and height 16 inches.
3. Part (b) makes the name list and salary list from the Employee_data dataframe for plotting. Further, this creates the first subplot – a bar graph of employee name and their salary.
4. Part (c) contains the code for finding the maximum salary, which is followed by creating the data of the employees (*Maximum_Sal_emp_data*) who are drawing the maximum salary. Please note that more than one employee may draw the maximum salary.
5. Part (d) creates annotations for all the employees who are drawing the maximum salary using the index named (*ind_max_sal*). The for loop annotates the entire list (see Figure 9). Please note the location of the text to be displayed (“Maximum Salary”), as given in xytext (-50,0), is in pixels. This section also adds the title and axis labels.
6. Part (e) creates the second subplot, a line graph of department-wise total salary. In addition, this graph is annotated for the department drawing minimum total salary. Please note in the annotation only one point is annotated and use of f-format in defining the label to be printed (*f'Minimum: {min_sumsal}'*). Please note this command has created the label *Minimum: 35000*, as *min_sumsal* is computed to be 35000 for the ‘Design’ department (refer to Figure 9). Finally, it also prints text “Total Salary” in the middle of line indicated by the command: *subplot[1].text(0.5, 0.5, "Total Salary", fontsize=10, transform=subplot[1].transAxes)*.

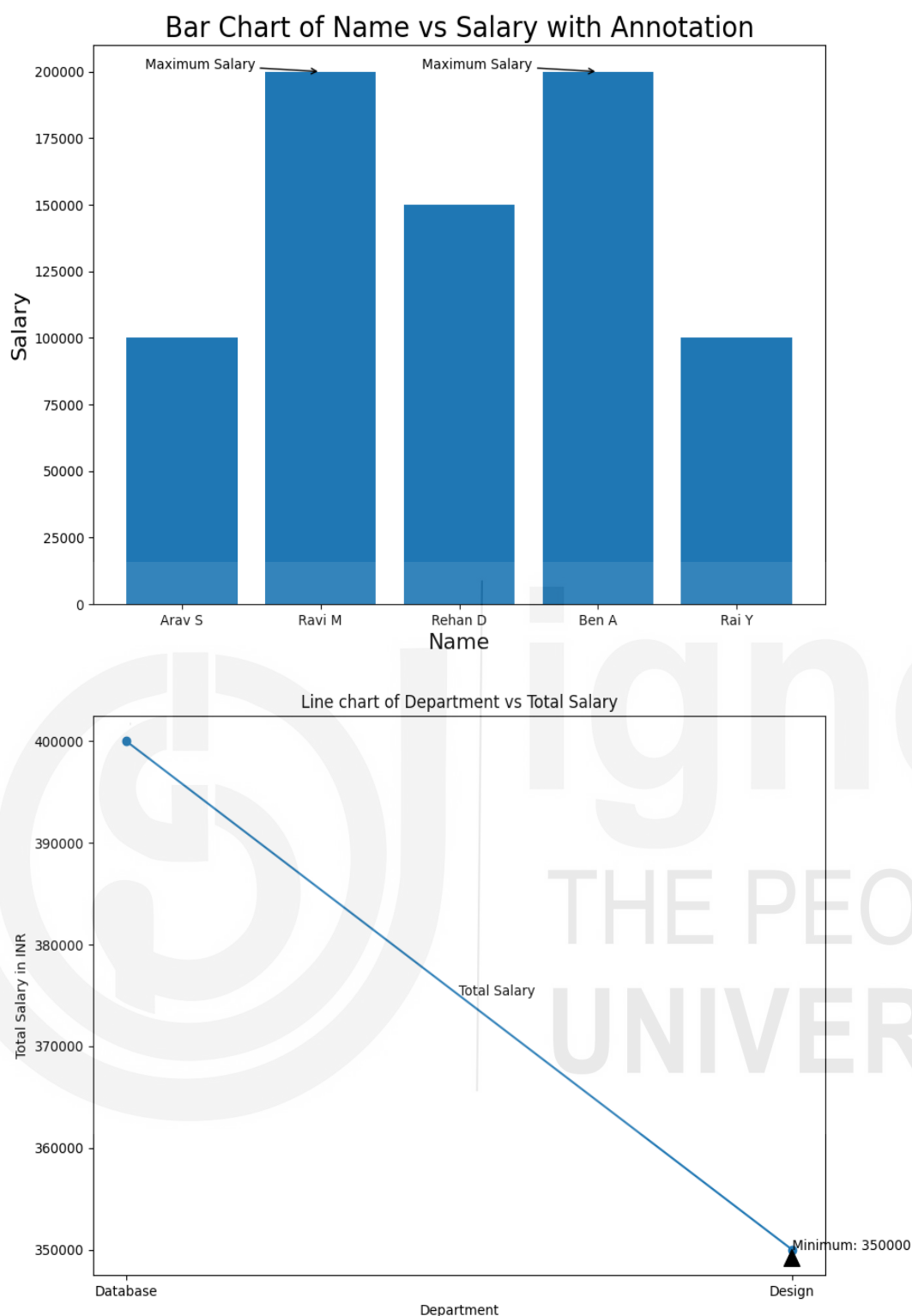


Figure 9: The output of the Program of Figure 8

6.4 SAVING PLOTS TO A FILE

Graphs are visual data. They are drawn to represent certain characteristics of data so that users and decision makers can interpret them easily. Therefore, plots are to be shared with decision makers and other stakeholders. This requires graphs to be saved in a format such that they can be included in different reports. Python allows plots to be saved in different standard image formats, like Portable Network Graphics (.png), Portable Document Format (.pdf), Joint Photographic Experts Group (.jpg) and Scalable Vector Graphics (.svg). It may be noted that *png* files are made up of pixels (called raster

images), *pdf* is a format of documents in which both text and graphics are included, *svg* files are files which store vector graphics and *jpeg* is a standard that uses lossy compression.

```
import pandas as pd
# Part (a): Code to Create the data frame Employee_data
empdata = {'Name': ['Arav S', 'Ravi M', 'Rehan D', 'Ben A',
                    'Rai Y'],
            'Department': ['Design', 'Database', 'Design',
                           'Database', 'Design'],
            'Specialisation': ['Web Development', 'SQL', 'SQL', 'Web
                               Development', 'SQL'],
            'Salary': [100000, 200000, 150000, 200000, 100000],
            'YearsWorking': [3.5, 2.5, 1.5, 2.0, 1.0]}
#Creating a data frame of the data of employees
Employee_data = pd.DataFrame(empdata)
```

```
#Part (b): Create two canvases - one each for each plot of 8
inches width by 4 inches length
import matplotlib.pyplot as plt
canvas1, plot1 = plt.subplots(figsize=(8, 4))
canvas2, plot2 = plt.subplots(figsize=(8, 4))
```

```
#Part (c): Prepare the data to create the bar plot in plot1
with Title and axis Labels
name_list = Employee_data['Name'].tolist()
salary_list = Employee_data['Salary'].tolist()
plot1.bar(name_list, salary_list)
plot1.set_title("Name vs Salary", fontsize=12)
plot1.set_xlabel("Name", fontsize=10)
plot1.set_ylabel("Salary", fontsize=10)
```

```
# Part (d): plot2 is a line chart between Department and
Total Salary
GroupOnDept=Employee_data.groupby('Department')
TotalSalFrame=GroupOnDept['Salary'].sum()
department_list = TotalSalFrame.index.tolist()
sumsalary_list = TotalSalFrame.values.tolist()
#Draw the plot2 with Title and Axis Labels
plot2.plot(department_list, sumsalary_list, marker='o')
plot2.set_title("Department vs Total Salary")
plot2.set_xlabel("Department")
plot2.set_ylabel ("Total Salary in INR")
plot2.set_facecolor('lightyellow')
```

```
#Part (e): Saving plot1
#Saving plot1 in jpg file at 200 dpi
canvas1.savefig("NameandSal.jpg", dpi=200)
#Saving plot1 in pdf file
canvas1.savefig("NameandSal.pdf")
#Saving plot1 in svgfile
canvas1.savefig("NameandSal.svg")
```

```
#Part (f): Saving plot2
#Saving the plot2 as a png file with DPI of 200
canvas2.savefig("DeptandSal.png", dpi=200)
#Saving plot2 as a png file with Transparent Background
canvas2.savefig("NobackDeptandSal.png", transparent=True)
```

Figure 10: Program to Save Figures in different File formats

Figure 10 shows a program to save plots in different file formats. It performs the following operations:

1. Part (a) creates the dataframe named `Employee_data`
2. Part (b) creates two independent canvases, named `canvas1` and `canvas2`, both of size 8 inches width and 4 inches height.
3. Part(c) creates `plot1` on `canvas1`, by first preparing the data and then plotting the bar chart with suitable title and axis labels.
4. Part(d) creates `plot2` on `canvas2`, by first preparing the data and then plotting the line chart with suitable title and axis labels. Please note the last statement in this part (`plot2.set_facecolor('lightyellow')`), which sets the background colour of `plot2`.
5. Part(e) saves the `plot1` in three different file formats, viz. `jpg` with a resolution of 200 dots per inch (dpi), `pdf` and `svg`. The output of this part contains three files named `NameandSal.jpg`, `NameandSal.pdf`, and `NameandSal.svg`, respectively. All these files contain the bar chart as shown in Figure 11.
6. Part(f) saves the file using `.png` format with 200 dpi, initially with a background colour and then with a transparent background. The two files created by this part are: `DeptandSal.png` and `NobackDeptandSal.png`. The content of these files are shown in Figure 12(a) and Figure 12(b) respectively.

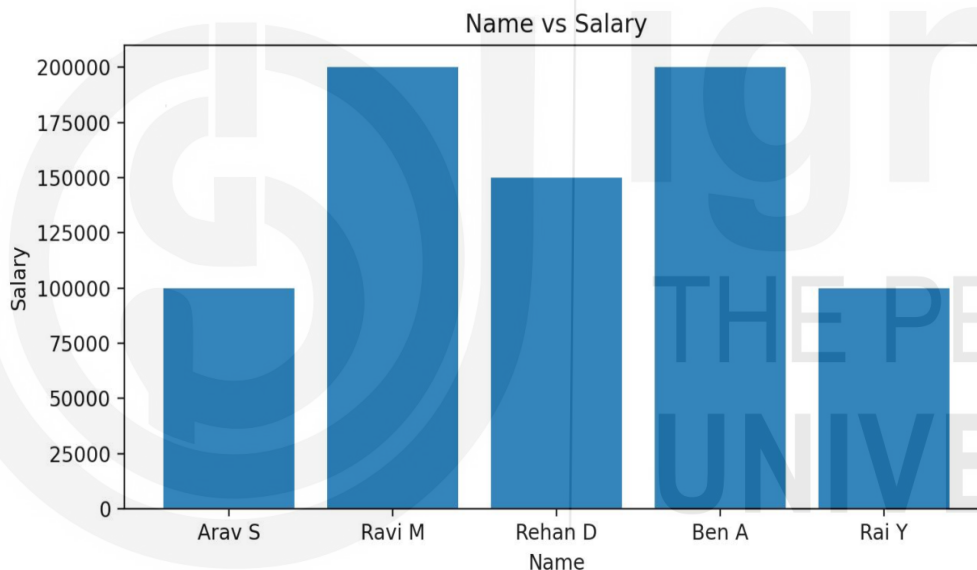


Figure 11: Plot contained in .jpg, .pdf and .svg files

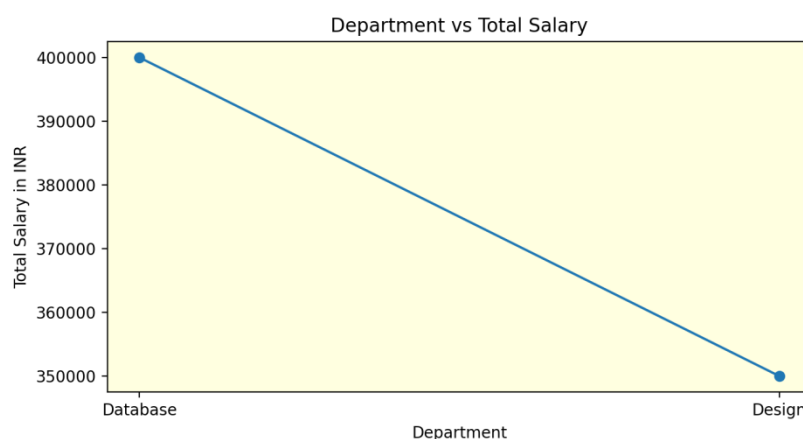


Figure 12 (a): plot2 as png file with background colour

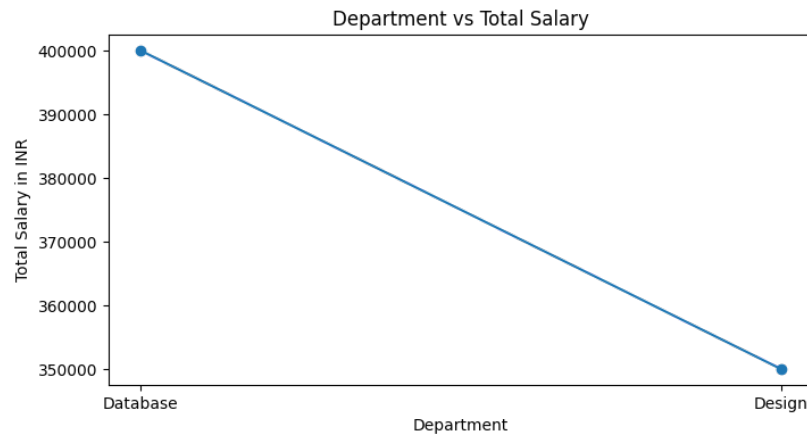


Figure 12 (b): plot2 as png file with a transparent background

Check Your Progress 2

1. How can you change line colour, line marker, ticks and legends in a plot?

.....

.....

.....

2. What are annotations in a graph? How can you create them?

.....

.....

.....

3. List the file formats in which you can save a plot. What function is used for this purpose?

.....

.....

.....

6.5 USE OF CONFIGURATION IN MATPLOTLIB

Configurations are an important part of matplotlib library that allows you to set a consistent look and feel for your visualisations. Matplotlib library also supports standard configurations. The matplotlib library uses a dictionary, named *rcParams*, to store the default configurations related to plots. You can set different parameters in this file using this dictionary. For example, the following code will set the font size of the axis labels in a figure to 14 points.

```
plt.rcParams['axes.labelsize'] = 14
```

Alternatively, you can use a function *plt.rc()* to change parameters. For example, to change the line width to size 2 and to change the marker to a square marker, you may use the configuration function call as:

```
plt.rc('lines', linewidth=2, marker='s')
```

You may change the axes labels to 14 point size and title size to 16 points and to set the background colour of graph to light blue, you may use the following function call:

```
plt.rc('axes', labelsz=14, titlesz=16, facecolor='lightblue')
```

Matplotlib also has a list of predefined styles, which can be used for specific style configuration. For example, one of the popular style is *ggplot*. It can be used by using the following command:

```
plt.style.use('ggplot')
```

You can set a custom style layout of your plots using the configuration. A detailed discussion on this topic is beyond the scope of this unit. You may refer to any Python documentation for more details.

6.6 PLOTTING WITH PANDAS AND SEABORN

In addition to matplotlib library, you can plot with pandas and seaborn libraries. The following are some of the functions that you can use to make plots.

The pandas library has a `plot()` function that can be used to plot a graph and seaborn library can be used for advanced statistical plots. For example, program given in Figure 13 uses pandas `plot()` function to draw the first plot and then uses seaborn library bar chart function to plot department and average salary of that department.

```
#Part (0) import different libraries
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
```

```
#Part (a): Code to Create the data frame Employee_data
empdata = {'Name': ['Arav S', 'Ravi M', 'Rehan D', 'Ben A',
' Rai Y'],
'Department': ['Design', 'Database', 'Design', 'Database',
'Design'],
'Specialisation': ['Web Development', 'SQL', 'SQL', 'Web
Development', 'SQL'],
'Salary': [100000, 200000, 150000, 200000, 100000],
'YearsWorking': [3.5, 2.5, 1.5, 2.0, 1.0]}
#Creating a data frame of the data of employees
Employee_data = pd.DataFrame(empdata)
```

```
#Part (b): Create two canvases - one each for each plot of 8
inches width by 4 inches length
canvas1, plot1 = plt.subplots(figsize=(8, 4))
canvas2, plot2 = plt.subplots(figsize=(8, 4))
```

```
#Part (c): Make the line plot in plot1 using pandas' plot()
Employee_data.plot(kind='line', x='Name', y='Salary',
marker='o', ax=plot1, legend=False)
#Add the Title, axis labels, and change the face colour
plot1.set_title("Name vs Salary")
plot1.set_xlabel("Name")
plot1.set_ylabel("Salary in INR")
plot1.set_facecolor('lightyellow')
```

```
#Part (d): plot2 is a bar chart on categorical data -  
Department, against the average salary of each department.  
Created using the seaborn library  
sns.barplot(data=Employee_data, x="Department", y="Salary",  
ax=plot2, errorbar=None)  
plt.title("Average Salary of Each Department using Seaborn")  
#Display the two plots  
plt.show()
```

Figure 13: Plotting pandas function plot() and seaborn Library

The Part(0) in Figure 13 imports different libraries including matplotlib, pandas, numpy and seaborn. Part (a) creates the dataframe named Employee_data. Part (b) creates the canvas and defines the sizes of the graph. Part (c) uses the plot function, specifying that it is a *line* chart between *Name* and *Salary* columns. The name of the plot is *plot1* and it does not have a legend. The type of markers on the line chart are also specified. This statement is followed by the statements that set the title, axis labels and face colour. Part (d) uses the seaborn library to create a bar plot on categorical data *Department* and *Salary*. Please note that it automatically calculates the average salary of each department and displays it as a bar. The output of this command is shown in Figure 14.

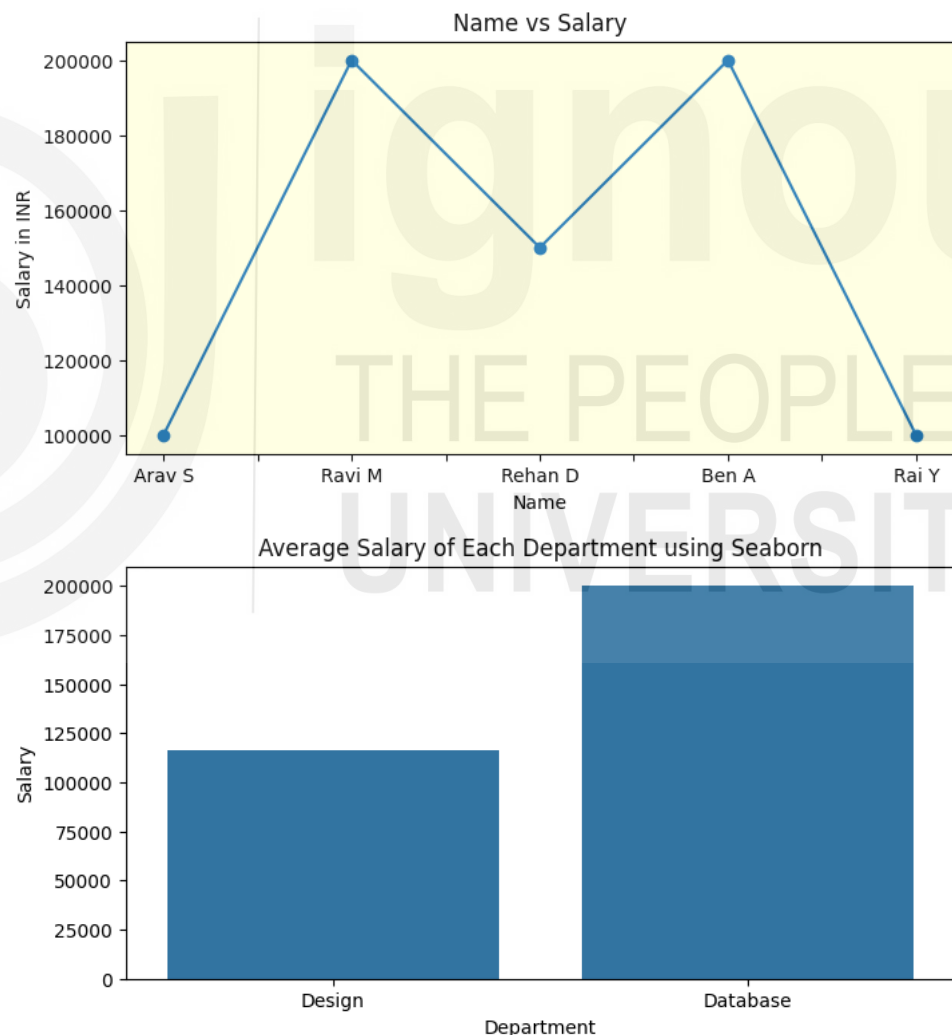


Figure 14: Output of Program of Figure 13

Thus, seaborn supports a rich library of plots, whereas pandas uses the plot() function to make plots using matplotlib. The seaborn library is made from the matplotlib library only. Thus, all three libraries can be integrated, as shown in the program of Figure 13.

6.7 MAKING DIFFERENT TYPES OF PLOTS

As discussed earlier, plots are very important visualisation tools. Python has several libraries that help in making plots. Here, we demonstrate the process of creating some useful plots with the help of Python. The following program code segment imports all the libraries that we will use for creating plots. Further, it creates a dataframe and lists for plotting the data.

```
# Part (0): Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
#Part(a): Code to Create the dataframe and lists for plotting
revenue = {'StoreZone': ['East', 'West', 'North', 'South',
'Central'], 'SDLC': ['Web Development', 'Database', 'Database',
'Web Development', 'Testing'], 'RevenueOfMonth': [500000,
800000, 650000, 900000, 700000], 'Expenses': [350000, 600000,
250000, 500000, 600000]}
#Creating a data frame of the Revenue Data
Revenue_data = pd.DataFrame(revenue)
revenueandExpensesLabels = Revenue_data['StoreZone'].tolist()
revenue_list = Revenue_data['RevenueOfMonth'].tolist()
expense_list = Revenue_data['Expenses'].tolist()
```

Figure 15(a): Importing libraries and preparing data for plotting

6.7.1 Line Plots

Line plots are one of the common plots and are used to visualise and compare data values. We have already made a line plot in the previous section. In this section, we demonstrate making of a multiple line plot. The following code draws a multiple line chart:

```
#Part(b) Multiple Line plot
plt.figure(figsize=(8, 4))
plt.plot(revenueandExpensesLabels, revenue_list,
label="Revenue")
plt.plot(revenueandExpensesLabels, expense_list,
label="Expenses")
plt.title("Zone wise Revenue and Expenses: Line plot")
plt.xlabel("Zone")
plt.ylabel("Amount in INR")
plt.legend()
plt.show()
```

Figure 15(b): Code for creating Multiple Line chart

Please note that code first creates a figure area of 8 inches width and 4 inches height, it is followed by two plot method calls, both creating one line each. (See Figure 16(a)). Rest of the code is self explanatory.

You may observe in Figure 16(a) that North zone has minimum expenses. Also, South zone has maximum revenue and the profit (revenue – expenses) is minimum in the central zone.

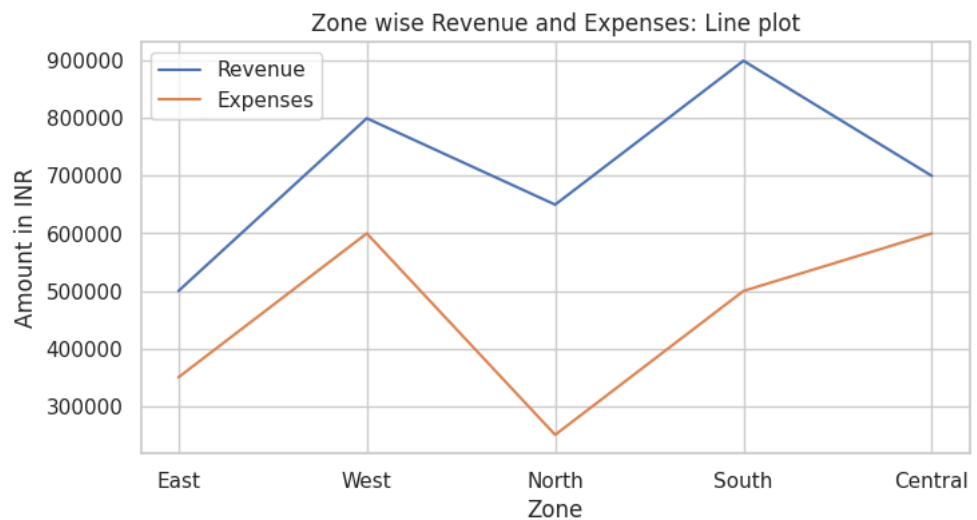


Figure 16(a): Multiple Line chart

6.7.2 Bar Plots

Bar plots are another useful plot for visualising and comparing data. In this section, we have created a grouped bar chart. This is also a very interesting chart.

```
#Part(c): Grouped Bar plot
plt.figure(figsize=(8, 5))
x_pos = np.arange(len(revenueandExpensesLabels))
width = 0.35
plt.bar(x_pos - width/2, revenue_list, width, label='Revenue')
plt.bar(x_pos + width/2, expense_list, width, label='Expense')
plt.xticks(x_pos, revenueandExpensesLabels)
plt.title("Zone wise Revenue and Expenses: Grouped Bar plot")
plt.xlabel("Zone")
plt.ylabel("Amount in INR")
plt.legend()
plt.show()
```

Figure 15(c): Code for making Group Bar chart

First, you prepare a figure of size 8 inches in width and 4 inches in height. Next, the x-axis positions are arranged by counting the number of labels that are to be displayed on x-axis. The width of a bar is then decided (0.35 inch in this case). Since two bars – one for the Revenue and one for the Expenses are to be plotted, the position on the x-axis is adjusted by half the width so that the bar is displayed on both sides of the x-position of the first point. The ticks that are displayed on x-position are taken from the `revenueandExpensesLabels` list created earlier. This is followed by display of titles and legends. Figure 16(b) shows this chart.

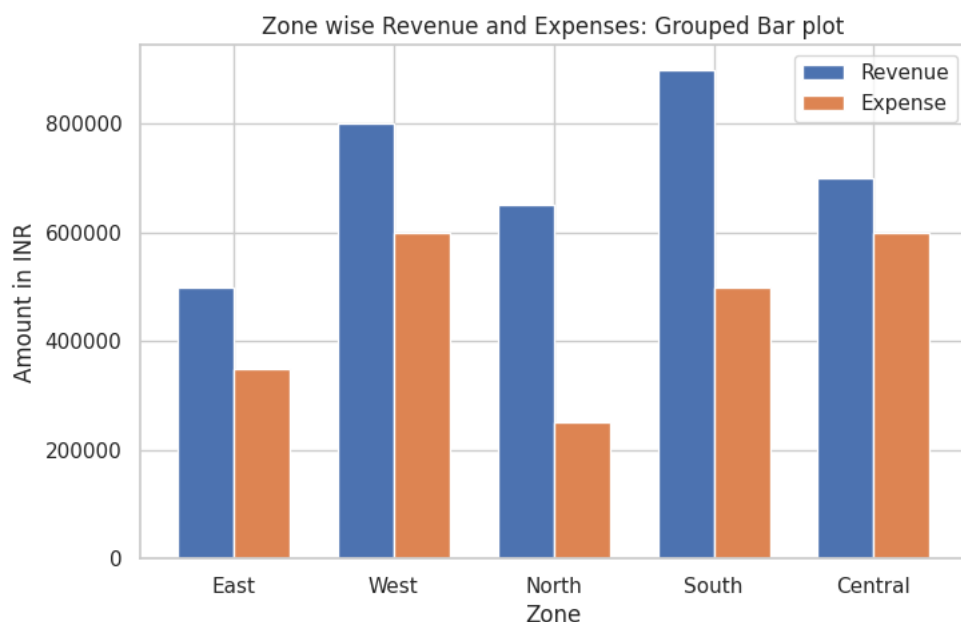


Figure 16(b): Grouped Bar Chart

6.7.3 Histograms and Density Plots

Histogram shows the grouped frequency of numerical data and are useful to observe if data is close to normal distribution or not. Density plot shows the approximate probability distribution of a continuous data. The following code plots the histogram and density plot.

```
#Part (d): Histogram
plt.figure(figsize=(4, 2))
plt.hist(revenue_list, bins=3)
plt.title("Histogram of Revenue with 3 bins")
plt.xlabel("Range") # Label for the x-axis
plt.ylabel("Frequency")
plt.show()
```

```
# Density Plot
plt.figure(figsize=(4, 2))
sns.kdeplot(revenue_list, fill=True)
plt.title("Density plot of Revenue")
plt.xlabel('Value')
plt.ylabel('Probability Density')
plt.show()
```

Figure 15(d): Code for making Histogram and Density Plot

Please note that we have specified the number of bins for the histogram. The x-axis shows the data and y-axis shows the frequency of the data related to the bin. Figure 16(c) shows the Histogram of the data shown in Figure 15(a). Please observe the number of bars are same as the number of bins specified by us.

The density plot has been drawn using the seaborn library. It shows probability density on the y-label.

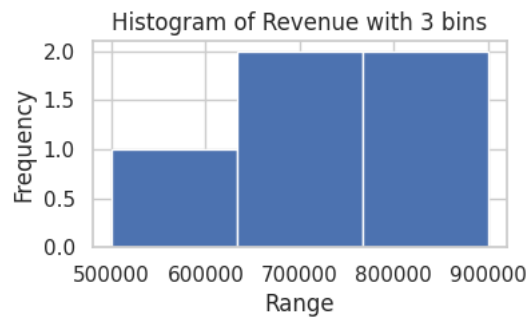


Figure 16(c): Histogram with bins

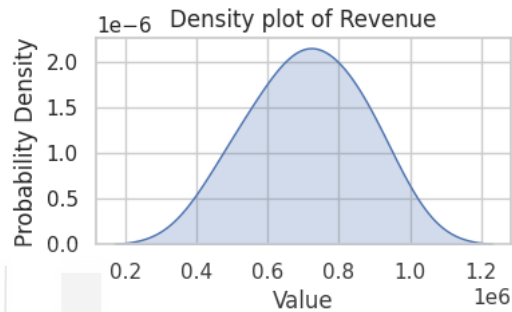


Figure 16(d): Density plot chart

6.7.4 Scatter or Point Plots

Scatter plots are plots between two numerical variables. They are very useful to determine if there is any relationship between two numerical variables. Python also allows for fitting a regression line in the data. Figure 15(e) shows the code for fitting a regression line between Revenue and Expenses.

```
#Part(e): Scatterplot with regression line
plt.figure(figsize=(4, 2))
sns.regplot(x=revenue_list, y=expense_list)
plt.title("Scatter Plot with Regression line")
plt.xlabel('Revenue')
plt.ylabel('Expenses')
plt.show()
```

Figure 15(e): Code for Creating Regression line

The code given in Figure 15(e) is self-explanatory. Its output is shown in Figure 16(e).

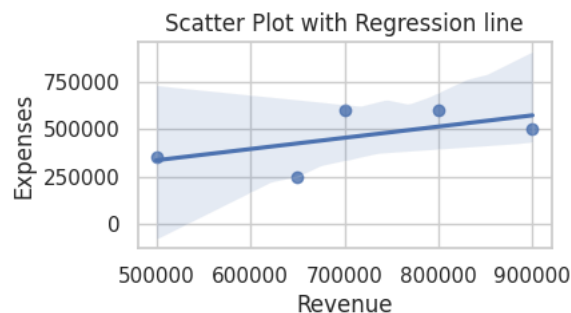


Figure 16(e): Scatter plot with Regression line.

Facet grids are used to categorise data on some categorical value. The objective is to study the features or distribution of data for specific categories. Figure 15(f) contains the code for creating a facet grid on the category *SDLC* and draw the scatterplot for each of the categories.

```
#Part (f): Use of Facet Grid
# Create facet grid on SDLC
facetgrid1 = sns.FacetGrid(Revenue_data, col="SDLC")
# Draw scatter plot using seaborn Library
facetgrid1.map(sns.scatterplot, "RevenueOfMonth", "Expenses")
plt.show()
```

Figure 15(f): Creating a Facet Grid and Using it for drawing Scatter Plot

The code of Figure 15(f) will first create a facet grid using seaborn on the “SDLC” column. Since our data consists of three different SDLC phases, viz., *Web Development*, *Database* and *Testing*, the scatter plot statement will draw three plots, one each for each *SDLC*. Please note that the *map* function is directly taking data from the dataframe; therefore, the dataframe column names are to be used as parameters. The output of this code is shown in Figure 16(f).

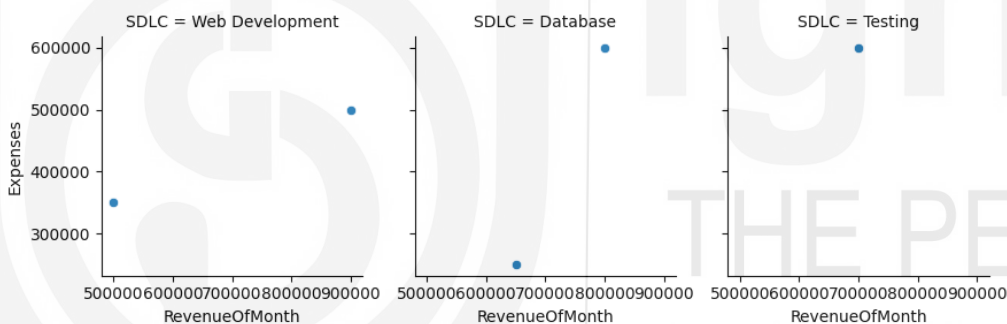


Figure 16(f): Scatter plots based on Facet Grid on SDLC

There are many other kinds of plots and many more options while making a graph. However, those are beyond the scope of this Unit. You may refer to this course for more details on those.

Check Your Progress 3

1. Why are configurations used in matplotlib? Name one popular configuration style?

.....

.....

.....

2. Why do you use pandas and seaborn libraries?

.....

.....

.....

3. Why do you need to make a scatter plot and facet grid?

.....

.....

.....

6.8 SUMMARY

This unit presents a brief introduction to plotting and visualisation using Python programming language. It has covered the following main points:

- It highlighted the need of data visualisation. It explained the use of matplotlib library of Python for plotting.
- The coding for creating charts with titles, axis-labels, legends, annotations, etc. has been discussed.
- It discusses the way of creating more than one plot in a single figure.
- It explains the process of saving plots as a file in several picture formats like png/ pdf etc.
- It highlights importance of using configurations.
- It also introduces the use of seaborn library for plotting.
- Finally, the unit discusses the code of creating plots like line plot, bar plot, histogram, scatter plot and facet grids.

6.9 SOLUTIONS / ANSWERS

Check Your Progress 1

1. Use the following commands

```
import matplotlib.pyplot as plt #import the pyplot module of matplotlib as plt
#assume data is in list1 and list2
plt.bar(list1, list2)          #plot the bar chart
plt.show()                    # show it on the specified output
```

2. Use the following steps

- i) The first step here would be to import the *pyplot* module of matplotlib library
 - ii) Create an overall figure, we have named it canvas in Figure 4, which will contain the objects named subplots in Figure 4.
 - iii) You can create a matrix of subplots, like in Figure 4, four subplots as 2 by 2 matrix would be displayed.
 - iv) You should plot each subplot of this matrix using matrix indexes, [0,0], [0,1], [1,0] and [1,1]
 - v) Finally, show this figure.
3. Different kinds of charts require different kinds of axis labels. For example, in pie chart, you do not have axes rather, you make data labels, whereas in line graph, you draw labels for x-axis and y-axis. You may create a list of labels and then use them with functions like *set_xlabel* and *set_ylabel* etc. You can draw title of the graph using function *set_title*.

Check Your Progress 2

1. You can change line colour, line marker, ticks and legends in a plot by specifying parameters in a plot function. You can set colour by using *color*, change style of a line using *linestyle*, change colour and size of the marker

on a line using *markerfacecolor* and *markersize* parameters respectively. For setting ticks, first you need to use *minorticks_on* function to display minor ticks. You can then set the parameters like length of major and minor ticks using *tick_params* function. Legends can be displayed using *legend* function.

2. Annotations are description given on the graph to explain a particular content. For example, in Figure 9, the maximum salary is shown through annotation. Annotations can be created using *annotate* function. You can specify a number of parameters in this function, as shown in code given in Figure 8.
3. You can save a plot as .png, .pdf, .jpg, .svg and several more formats. The function *savefig* with appropriate parameters can be used to save a plot in a file format of your choice.

Check Your Progress 3

1. Configurations help in creating a standard format in which you can draw all your plots. Thus, you can create a consistent style of plots. One of the popular configuration is *ggplot*.
2. pandas library uses a *plot* function for plotting. This function along with parameters can simplify plotting process. The seaborn library is built over matplotlib and provides a very rich set of customised plots.
3. Scatter plot can be used to display relationship between two numerical variable. Facet grid can help you study the distribution of data for a specific category.